# API

## 1 - O que é uma API ?

API é um **software que se comunica com outro software**. Ela funciona como uma ponte ou um "garçom" que leva o pedido de um aplicativo e traz a resposta de um servidor.

### Os 4 Comandos Principais (Verbos HTTP)

Para essa comunicação acontecer, usamos quatro comandos básicos:

- **GET (Pegar):** Você pede para a API te entregar uma informação que já existe.
- **POST (Enviar):** Você envia novos dados para a API salvar ou processar.
- **PUT (Atualizar):** Você envia uma informação para substituir ou corrigir algo que já está lá.
- **DELETE (Apagar):** Você dá uma ordem para a API remover um dado específico.

### Quem usa o quê?

1. **O Usuário Comum:** Geralmente, ele só "mexe" no **GET**. Quando você abre um app de clima ou de cotação de Bitcoin em Joinville, você está apenas pedindo (GET) para ver os dados.
2. **A Empresa/Desenvolvedor:** É quem está "atrás das cortinas". A empresa usa o **POST, PUT e DELETE** para gerenciar o sistema, cadastrar novos preços, atualizar informações e limpar o banco de dados.

# Explicação do Código

📂 Caminho: Models/ProdutoFinanceiro.cs

Criação do Modelo de Armazenamento da API

```
Asp.Net_Projeto01_WebAPI_Base > Models > ProdutoFinanceiro.cs > ...
1   namespace Asp.Net_Projeto01_WebAPI_Base.Models;
2
    4 referências
3   public class ProdutoFinanceiro
4   {
        0 referências
5       public int Id { get; set; }
        0 referências
6       public string Nome { get; set; } = string.Empty;
        0 referências
7       public decimal Preco { get; set; }
        0 referências
8       public string Tipo { get; set; } = "Ação"; // Ex: Cripto, FII, ETF
9   }
10
```

📂 Caminho: `Controllers/InvestimentosControllers.cs`

```
Asp.Net_Projeto01_WebAPI_Base > Controllers > InvestimentosController.cs > ...
1   using Microsoft.AspNetCore.Mvc;
2   using Asp.Net_Projeto01_WebAPI_Base.Models; // Importante para enxergar o Model
3
4   namespace Asp.Net_Projeto01_WebAPI_Base.Controllers;
5
6   [ApiController]     // Indica que essa classe é um Controller de API (faz com que o ASP.NET entenda que ela é responsável por responder a requisições HTTP)
7   [Route("api/[controller]")]     // Define a rota usando o nome do controller (http://localhost:5231/api/Investimentos)
    0 referências
8   public class InvestimentosController : ControllerBase
9   {
10      // Lista que simula um banco de dados
        2 referências
11      private static List<ProdutoFinanceiro> _bancoDeDados = new List<ProdutoFinanceiro>();
12
13      [HttpGet]   // Indica que é uma requisição Get
        0 referências
14      public ActionResult<List<ProdutoFinanceiro>> ListarTodos()
15      {
16          return Ok(_bancoDeDados);
17      }
18
19      [HttpPost]  // Indica que é uma requisição Post
        0 referências
20      public ActionResult Criar(ProdutoFinanceiro novoProduto)
21      {
22          _bancoDeDados.Add(novoProduto);
23          return Ok("Produto cadastrado com sucesso!");
24      }
```

### 1. Configurações Globais (`[ApiController]` e `[Route]`)

- `[ApiController]`: Ativa as configurações automáticas de API, como a validação de dados e a resposta padronizada de erros.
- `[Route("api/[controller]")]`: Define que o endereço para acessar esses métodos será `api/investimentos`. O .NET remove a palavra "Controller" do nome da classe automaticamente para criar a rota.

### 2. Herança (`ControllerBase`)

- **O que é**: Sua classe herda de `ControllerBase` para ganhar acesso a métodos como o `Ok()`, que envia a resposta de sucesso para quem pediu o dado. Como é

uma API pura, não usamos a classe `Controller` (que serve para sites com telas) para manter o projeto mais leve e profissional.

### 3. Simulação de Banco de Dados (`private static List`)

- `List<ProdutoFinanceiro>`: Uma lista que armazena os objetos criados a partir do seu Model.
- `static`: Esta palavra é fundamental aqui. Ela garante que a lista seja a mesma para todos os usuários e que os dados não sumam cada vez que você faz uma nova chamada na API. Os dados só serão apagados se você parar o projeto.

### 4. O Método GET (`[HttpGet]`)

- **Ação**: "Pegar dados".
- **Retorno**: Ele retorna um `ActionResult`, que é uma resposta HTTP contendo a sua lista de investimentos. Se a lista estiver vazia, ele retorna `[]`. Se tiver dados, ele envia o JSON completo.

### 5. O Método POST (`[HttpPost]`)

- **Ação**: "Enviar coordenadas".
- **Parâmetro (`ProdutoFinanceiro novoProduto`)**: Aqui o .NET faz a "mágica" de ler o JSON que você enviou no Swagger e transformá-lo automaticamente em um objeto C# para ser adicionado à lista.

💡 Resumo do Fluxo

1. O **POST** envia o objeto para o `_bancoDeDados`.
2. O **GET** lê o que está dentro do `_bancoDeDados` e exibe na tela.

📂 `Caminho: Program.cs`

```
var builder = WebApplication.CreateBuilder(args);

// 1. Adiciona o suporte para os seus Controllers
builder.Services.AddControllers();

// 2. Configura o Swagger (Necessário para a interface visual)
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// 3. Configura o ambiente de desenvolvimento
if (app.Environment.IsDevelopment())
{
    // Gera o arquivo JSON da API
    app.UseSwagger();
    // CRIA A INTERFACE VISUAL no endereço /swagger
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

**1. O Construtor (`WebApplication.CreateBuilder`)**

- **O que é:** Ele prepara o terreno para a aplicação começar.
- **Função:** Ele carrega as configurações do seu projeto (como as portas `5231` que vimos no terminal) e prepara o sistema de injeção de dependência.

**2. Configuração de Serviços (`builder.Services`)**

Aqui você diz ao .NET quais ferramentas ele deve "colocar na caixa" antes de ligar o motor:

- **`AddControllers()`**: Avisa ao servidor que ele deve procurar por classes que herdam de `ControllerBase` (como o seu `InvestimentosController`).
- **`AddSwaggerGen()`**: Ativa o gerador automático de documentação. Sem essa linha, o Swagger não saberia quais campos o seu `ProdutoFinanceiro` possui.

**3. O Pipeline (O caminho que o dado percorre)**

Esta parte do código define a ordem em que as coisas acontecem quando alguém chama a sua API:

- `if (app.Environment.IsDevelopment())`: Uma trava de segurança. Ela garante que a interface visual do Swagger só apareça enquanto você está programando. Em um servidor real de produção (da "empresa"), essa tela geralmente fica escondida.
- `app.UseSwaggerUI()`: O comando que cria a página colorida que você usou para testar.

**4. Middlewares de Segurança e Roteamento**

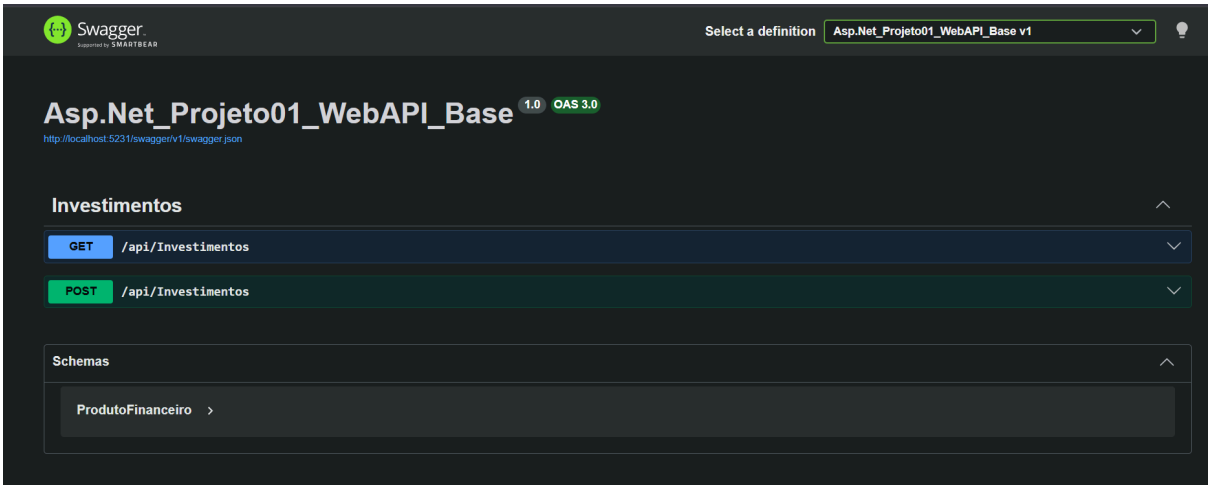
- `UseHttpsRedirection()`: Garante que a comunicação seja segura e criptografada.
- `MapControllers()`: É a "ponte" final. Ele liga as rotas que você definiu no Controller (o `api/investimentos`) ao motor de execução do .NET.

**5. O Início de Tudo (`app.Run()`)**

- **O que é:** O comando final que "tranca a porta" das configurações e liga o servidor de fato. A partir daqui, a API começa a "ouvir" as requisições na porta especificada.

🌎 URL: http://localhost:5231/swagger/index.html

Esta URL é o seu **Painel de Controle**. Como uma API não tem botões ou campos de texto próprios (Views), o Swagger "empresta" essa interface para que você possa agir como o cliente da sua própria API.



## Destaque Técnico e Explicação

### 1. O Cabeçalho (Título e Versão)

- **O que você vê:** O nome do seu projeto e a versão (v1).
- **Por que importa:** Em empresas grandes, uma API pode ter várias versões rodando ao mesmo tempo (v1, v2). O Swagger ajuda a garantir que você está testando a versão certa do seu sistema de investimentos.

### 2. A Lista de Endpoints (As "Portas de Entrada")

- **GET (Barra Azul):** Representa o seu método `ListarTodos()`. É aqui que você clica para pedir os dados ao servidor.
- **POST (Barra Verde):** Representa o seu método `Criar()`. É aqui que você envia o "pacote" com o novo investimento.

### 3. O Botão "Try it out" (O Modo de Teste)

- **Função:** Por padrão, o Swagger só mostra a documentação. Quando você clica em "Try it out", ele libera os campos para você digitar.
- **Dica:** É o equivalente a abrir um formulário em branco para preenchimento.

### 4. Request Body (Onde você coloca o JSON)

- **O que é:** É o espaço onde você digita as informações do seu `ProdutoFinanceiro`.
- **Destaque:** O Swagger é inteligente e já monta o modelo do JSON para você, baseado na sua classe `Models/ProdutoFinanceiro.cs`. Você só precisa trocar os valores.

### 5. O Botão "Execute" (O Envio Real)

- **Ação:** Ao clicar aqui, o Swagger faz uma chamada HTTP real para o seu servidor local.
- **O que acontece:** O dado sai do seu navegador, passa pelo `Program.cs`, entra no `InvestimentosController`, é adicionado à sua `static List` e o servidor devolve uma resposta.

### 6. Server Response (O Resultado)

Aqui é onde você vê se o seu software se comunicou bem:

- **Code (200):** Significa que tudo deu certo!
- **Response Body:** Mostra o que a API devolveu (a mensagem de sucesso no POST ou a lista de ativos no GET).